北京航空航天大学
科研课堂结题报告

基于 LoongArch 指令集的国产自主可控处理器设计

项目成员	安阳	22377264@buaa.edu.cn
	王琦	22376286@buaa.edu.cn
	刘栗江	21371345@buaa.edu.cn
指导老师	万寒	wanhan@buaa.edu.cn
	杨建磊	jianlei@buaa.edu.cn

2024 年 07 月 08 日

目录

1 | 绪论 4

1.1 研究目的及意义 4

1.2 相关研究 4

1.2.1 相关研究概述 4

1.2.2 LA32R 单发射处理器 4

1.2.3 LA32R 超标量处理器 5

1.2.4 “龙芯杯”中的 MIPS 处理器 5

1.3 报告结构 6

2 | CPU 设计理论 7

2.1 单周期 CPU 简介 7

2.2 流水线技术 8

2.2.1 流水线原理 8

2.2.2 分割流水线 9

2.2.3 流水线冒险 10

2.3 分支预测 11

2.3.1 分支预测原理 11

2.3.2 静态分支预测 12

2.3.3 动态分支预测 12

2.4 超标量处理器 12

2.4.1 为什么需要超标量 12

2.4.2 顺序执行的超标量处理器 13

2.4.3 乱序执行的超标量处理器 14

3 | CPU 核设计 15

3.1 CPU 核心概述 15

3.2 流水线结构概述 15

4 | SoC 设计及系统适配 17

4.1 SoC 设计 17

4.2 系统软件移植 18

5 | 评估及总结 20

5.1 Soc 仿真及上板情况 20

5.2 项目总结 21

参考文献 **22**

1 | 绪论

1.1 | 研究目的及意义

LoongArch（下称 LA 指令集）是龙芯中科推出的一款自主指令集架构，具有完全自主知识产权，它在设计上旨在提供高性能、高安全性和良好的兼容性。LA 指令集的推出打破了国外对指令集架构技术的垄断，为国内芯片设计提供了自主可控的技术基础。研究 LA 指令集，并设计基于 LA 指令集的处理器，有助于推动基于该指令集的软件生态建设，促进软硬件协同发展；同时从教育与人才培养上看，相关处理器结构的设计可以作为教学案例，促进计算机体系结构学科的发展，培养更多专业人才。

LoongArch 32 位精简指令集[1]（下称 LA32R 指令集）是标准 LA 指令集 32 位版本的精简版本，主要适用于高校教学和科研设计。LA 指令集是完全国产自主研发的主流指令集，拥有相当广泛的社区支持，而 LA32R 指令集也具有对诸如 Linux Kernel、GCC 和 Qemu 等软件的较完整支持。

目前国内高校一般计算机组成相关课程教学的一般要求为设计基于 MIPS 指令集的经典五级流水线 CPU。但考虑到自主研发 LA 指令集的出现，可以选择尝试设计 LA32R 指令集的 CPU，并进行简单的软件移植。

1.2 | 相关研究

1.2.1 | 相关研究概述

LA32R 指令集赛道在第七届“龙芯杯”全国大学生计算机系统能力竞赛中被引入，截至目前仅有一届举办，相关研究及参考资料较少。以下主要参考清华大学的 NOP 项目研究报告[2]，综述若干相关处理器设计。

1.2.2 | LA32R 单发射处理器

openLA500 处理器[3] 是由“龙芯杯”主办方开发并提供的单发射五级流水的示例 CPU，且具备了异常处理、虚实地址翻译功能，并附了简单的分支预测、指令/数据缓存等加速手段，作为相关 LA 赛道评测的基线处理器使用。相比于 MIPS 赛道的基线，LA 赛道的基线方法是十分完善的，其已具备了启动 Linux 的能力。

其它基于 LA32R 的单发射开源处理器还有“流云”处理器[4] 。它是一款基于 LA32R 指令集的单发射 7 级流水通用中央处理器核，对基线处理器做了进一步的流水段切分。

1.2.3 | LA32R 超标量处理器

“卅”处理器[5] 是一款基于记分牌算法（Scoreboard 算法）的动态调度乱序执行处理器，其使用两路算数执行单元、一路乘法执行单元、一路除法执行单元、一路访存执行单元，也支持了 TLB 等 LA 指令集规定的特权级功能，但其存在因记分牌算法导致的 WAR 和 WAW 冒险，进而有大量流水线阻塞现象的出现，无法有效开发出乱序的指令级并行。

“争流”处理器[6] 也是一款基于 LA32R 指令集的动态超标量处理器，其包括 8 个流水级，整个核分为前端，后端和访存子系统三部分，可仿真进行性能测试，并在仿真环境中启动 Linux 5.14 内核，但并未进行上板验证，且时序有显著不足。

NOP 处理器[2] 是第七届“龙芯杯”全国大学生计算机系统能力培养大赛参赛作品。NOP 处理器基于 LA32R 指令集设计，属于乱序多发射微架构，整体基于 Tomasulo 动态调度算法的思路 + 重排序缓存实现，并实现了分支预测、指令/数据缓存、数据旁路、推测唤醒等特性。从功能的角度上，NOP 实现了基础的算术指令、分支指令、访存指令，支持精确异常处理、虚实地址转换与 LA32R 指令集规定的各类中断，通过使用 PMON 引导程序，NOP 可以稳定地启动 Linux 操作系统。从性能上看，NOP 作为一款乱序多发射处理器，在大赛提供的 FPGA 实验平台与性能测试程序 / SoC 上达到了 107.69 MHz 的主频与 1.02 的 IPC。相比于基线 openLA500 处理器，NOP 核的整体加速比达到了 3.00，IPC 加速比达到了 1.402。同时，项目充分利用了大赛实验平台的板载硬件资源，设计了 NOP-SoC 系统以驱动板载外设，同时对相关系统软件的代码做了相应的修改适配，最终驱动起开发板上除 USB 外的全部外设。

1.2.4 | “龙芯杯”中的 MIPS 处理器

在第七届“龙芯杯”大赛前，由于只设 MIPS 赛道，所以有了大量基于 MIPS 指令集的高性能开源处理器设计的出现，在这里简单回顾。

2020 年的 NonTrivial-MIPS[7] 实现了具有 10 段流水段的双发射处理器，实现了异常处理机制、协处理器、可扩展的专用计算模块，可以启动 Linux 内核，其贡献的系统展示 SoC 设计与配套的 Linux 源码修改方式至今仍有很大的影响力。2021 年的 LLCL-MIPS[8] 实现了顺序双发射八级流水线的处理器，首创性地基于 SpinalHDL 进行开发，继承了 NonTrivial-MIPS[7] 对 VGA、PS/2、串口、Flash 等外设的支持，在大赛提供的开发板上频率达到了 120 MHz。2022 年的 ZenCove[9] 实现了五发射处理器，仿照 VexRiscv[10] 的设计模式实现了高度可配置的处理机核，在支持启动 Linux 的同时驱动起了开发板上除 USB 之外的全部外设，性能跑分超过了 NonTrivial-MIPS[7]。

上述处理器核均为基于 MIPS 指令集的处理器设计，但是由于 MIPS 和 LA32R 指令集的不同，如去掉了延迟槽的设计，不同的异常中断、虚拟地址转换逻辑等等，实际实

现过程中的细节上会有很大的出入。同时，如果需要软件适配，系统软件因为指令集的不同而需要重新配置，需要对 LA32R 版本的 PMON 和 Linux 内核进行相应的修改。

1.3 | 报告结构

第一部分为绪论，主要阐述了项目的目的和实际意义，介绍项目的基本信息和相关的研究工作。

第二部分主要结合若干次“龙芯杯”组织的讲座和相关资料，整理得到了部分 CPU 的设计理论基础。

第三部分主要介绍了 CPU 核心的设计细节和内容，包括流水线结构的设计细节等。

第四部分主要介绍了 SoC 的架构，如地址分布等，和部分系统软件在 LA32R 指令集上的移植。

第五部分主要评估了 CPU 的仿真和综合结果，并就整体项目研究过程给出一个总结。

2 | CPU 设计理论

2.1 | 单周期 CPU 简介

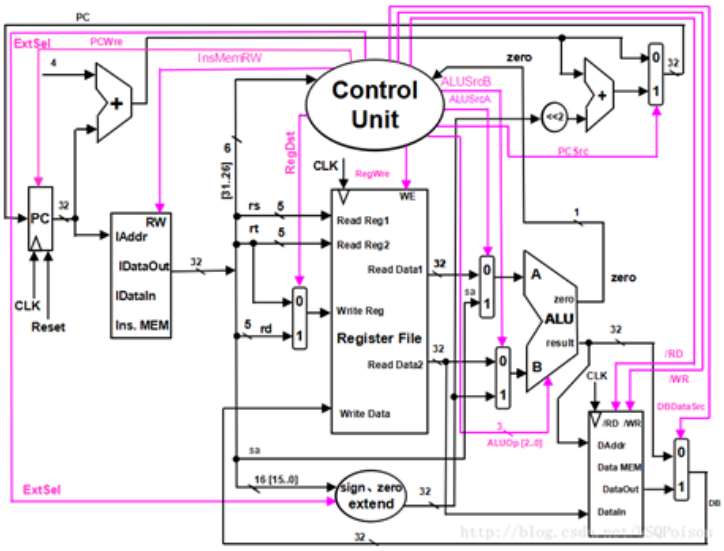

图 1　一个典型单周期 CPU 的数据通路

一个典型的单周期 CPU 示意图如图 1。对于其中组成元件的解释如下：

PC（程序计数器） 给出当前要执行的指令地址，同时获取下一条指令的地址。如果需要跳转，PC 根据传入的值获取下一条指令地址，否则自增 4 以获取下一条相邻指令。

InstructionMemory（指令存储器） 通过 PC 地址获取相应指令的。输入当前 PC 所指地址，输出对应地址的指令。

RegisterFile（寄存器堆） 大多数指令涉及寄存器，寄存器堆管理 32 个寄存器，同时可以获取不同寄存器的值，可以修改允许修改的寄存器的值。

- **输入**　RegWe（写使能信号），Clk（时钟信号），RegIn1（读寄存器的编号），RegIn2（读寄存器编号），WriteReg（写寄存器编号），WriteData（写入寄存器值）
- **输出**　RegOut1（读寄存器值），RegOut2（读寄存器值）

Mux（多路选择器） 由于 LA 指令格式的不同，进入寄存器堆的 WriteReg 端的可能是 rk 域，也可能是 rd 域，因此在这里要依据控制信号来选择寄存器。

SignZeroExtend（位扩展器） 指令中的 5/12/16/20 位立即数，要进入 ALU 的输入端必须扩展为 32 位，逻辑操作（与、或）使用的是无符号扩展；加、减等操作使用的是有符号扩展，根据信号选择无符号拓展或有符号拓展。

- **输入**：ImmNumber（指令中的立即数），ExtSel（符号扩展标识）
- **输出**：Result

ALU（运算单元） 负责的是对数据进行基本的算术操作，大多数操作都可能用到 ALU。

DataMemory（数据存储器） ld.w 和 st.w 等指令关系到数据的存储，数据存储器用于存放数据。

Decoder（解码器） 获取指令操作码并进行解析，发出相应的控制信号，指挥所有组件的工作。

2.2 | 流水线技术

2.2.1 | 流水线原理

一条指令的执行需要经过取指令，翻译指令，执行指令三个基本流程。CPU 内部的电路也分为不同的单元：取指单元、译码单元、执行单元等，指令的执行也是按照流水线的工序一步一步执行的。假设每个单元的工作周期都为 1，且不算流程转换的时间，若不采用流水线技术，则每个时钟周期内只有一个单元在工作，其余两个单元处于空闲状态，如图 2 。

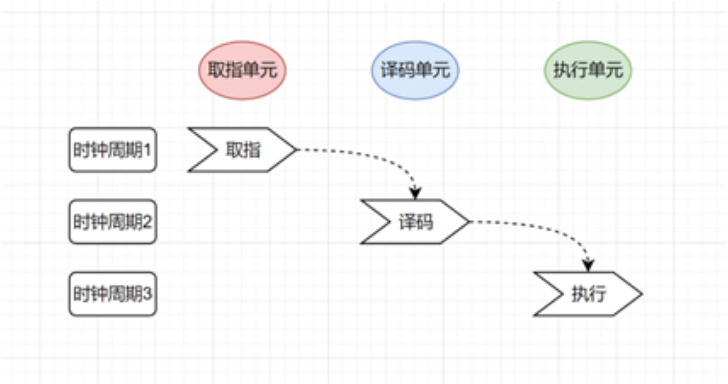

图 2　一个典型单周期 CPU 的时序图

为此引入流水线技术，使每个电路单元都状态拉满，而基本排除电路空闲。如图 3 ，对于流水线 CPU，从第三个时钟周期开始，每个单元都会满状态工作，CPU 的指令执行效率大幅提升。流水线的本质就是拿空间换时间。将每条指令的步骤分解到不同的电路单元，从而使得多个指令并行执行。

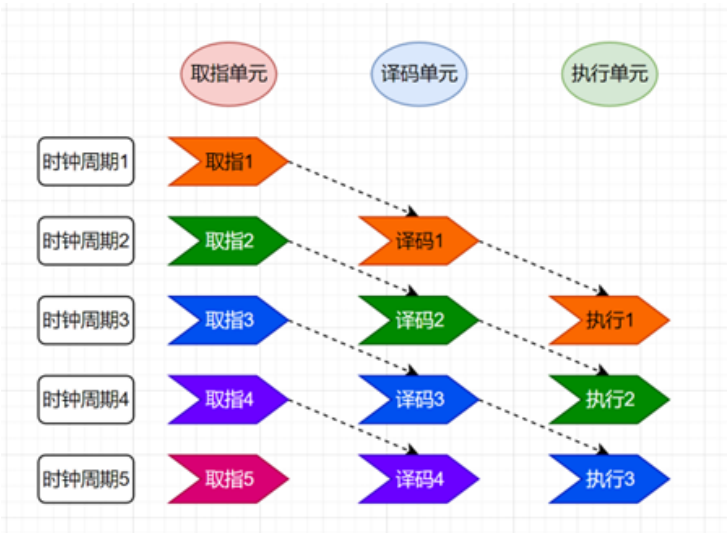

图 3　一个典型流水线 CPU 的时序图

2.2.2 | 分割流水线

流水线确实可以提升 CPU 的工作效率，但此处假设了每个小环节的执行周期是相同的。对于流水线每级用时不相同的情况，需要进一步讨论。假设取指单元的工作需要两个时钟周期来完成，分析时序如图 4 。

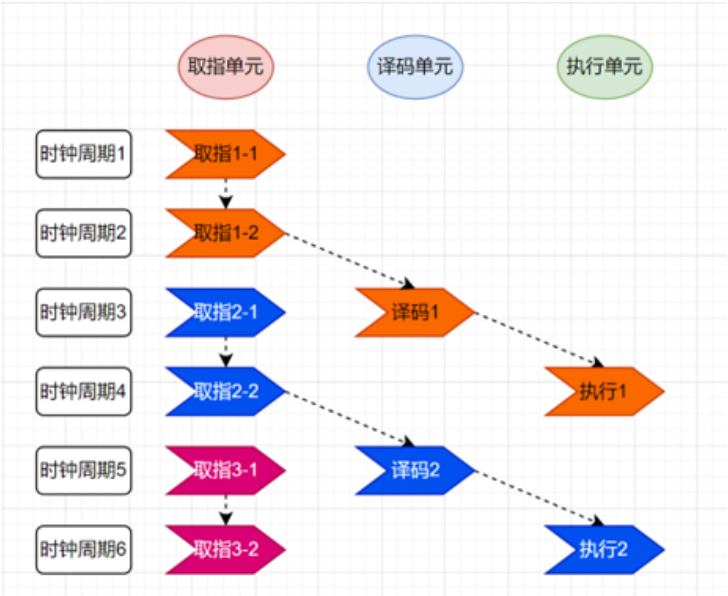

图 4　分割前流水线时序图

由图 4 可以发现，又会出现部分工作单元出现空闲的情况，为了进一步提升效率，考虑将取指的工作分配给两个物理单元去完成，这时又会像开始那样，非常高效地运行，如图 5 。

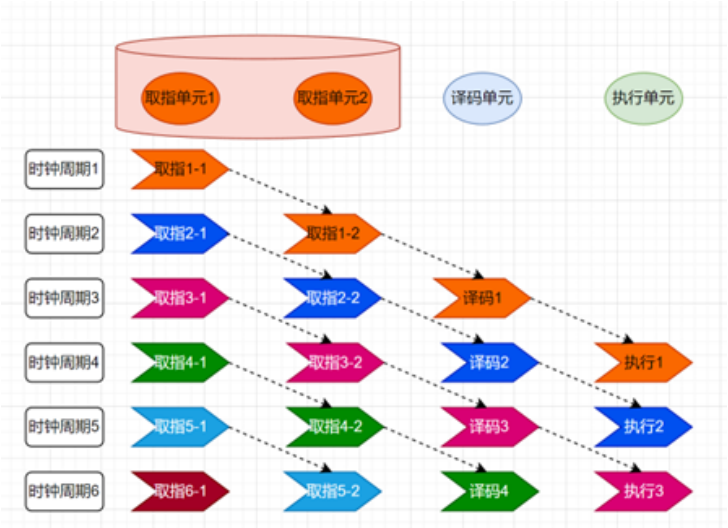

图 5　分割取指后流水线时序图

图 5 中的设计通过增加流水线的深度提升了 CPU 处理指令的效率。在优化前，CPU 执行一个指令平均需要 2 个指令周期；而优化后，CPU 执行一个指令平均仅需要 1 个指令周期。

实际的工程中，会根据实际情况来设计流水线的深度，通常把 5 级以上的流水线称为超流水线结构。高性能的处理器一般都会采用这种超流水线结构。Inter 的 i7 系列处理器有 16 级流水线，AMD 的速龙 64 系列 CPU 有 20 级流水线。

2.2.3 ｜流水线冒险

在计算机架构中，流水线的冒险是指在指令流水线的执行过程中，由于各种原因导致下一条指令无法按照预定的时钟周期正常执行，从而引起的性能问题。流水线冒险主要分为三种类型：结构冒险、数据冒险和控制冒险。

结构冒险　结构冒险是由于处理器资源冲突，导致某些指令组合无法在同一时钟周期内执行的情况。这通常发生在多个指令需要同时访问同一硬件资源（如内存、寄存器文件等）时，而该资源无法在同一时间满足所有指令的需求。在经典 LA 五级流水线中，如果指令和数据都存储在同一个存储器中，那么在指令获取（IF）阶段和内存访问（MEM）阶段可能会同时需要访问存储器，导致其中一个操作需要等待，从而引发结构冒险。同时读写存储器或同时读写寄存器也是常见的结构冒险情况。

数据冒险　数据冒险是指当前指令需要的数据正在被之前的指令操作，导致当前指令无法在预定的时钟周期内执行的情况。这通常发生在指令之间存在数据依赖关系时，即后一条指令需要前一条指令的执行结果作为输入。数据的三种相关性如下：

- RAW（Read After Write）：先写后读相关性。例如，如果第二条指令在第一条指令写回某个寄存器之前就读取该寄存器的值，就会引发 RAW 数据冒险。

- WAW（Write After Write）：先写后写相关性。如果两条指令都尝试写入同一个寄存器，且第二条指令在第一条指令写回之前就开始执行，就会引发 WAW 数据冒险。
- WAR（Write After Read）：先读后写相关性。在某些情况下，如果第二条指令在读取某个寄存器之后、第一条指令写回该寄存器之前就开始写入该寄存器，也可能导致逻辑错误，尽管这种情况相对较少见。

关于数据冒险，对应的解决方法有：

- 流水线停顿：在发生数据冒险时，暂停流水线的执行，直到所需的数据准备就绪。但这种方法会降低流水线的效率。
- 数据前递：在数据产生后立即将其传递给需要它的指令，而不是等待它写回寄存器。这种方法可以保持流水线的连续执行，提高效率。

控制冒险 控制冒险是指由于分支指令（如跳转、条件分支等）的执行结果尚未确定，导致下一条指令的地址无法立即确定，从而引发流水线停顿的情况。在执行条件分支指令时，如果分支条件尚未计算完成，但流水线已经按照顺序取出了下一条指令，那么当分支条件确定后，可能需要丢弃已经取出的指令并重新从新的地址取指令，这就会导致流水线停顿。对应的解决方法有：

- 分支预测：通过预测分支的方向来提前取出并执行可能的下一条指令。如果预测正确，则可以避免流水线停顿；如果预测错误，则需要丢弃已经执行的指令并重新从正确的地址开始执行。
- 延迟槽：在某些架构中，将分支指令的延迟槽（即分支指令之后、跳转目标之前的指令）用于执行与分支结果无关的操作，以减少分支误预测的影响。

流水线的冒险是计算机架构中需要关注的重要问题之一。通过合理的设计和优化策略，可以最大限度地减少冒险对流水线性能的影响。

2.3 | 分支预测

2.3.1 | 分支预测原理

条件跳转引起的控制冒险虽然也可以通过在流水线中插入空泡来避免，但是这毕竟不是最优解，因为还是较大程度降低了效率，流水线越深，就需要插入越多的空泡。假如相邻的两条指令引起了控制冒险，则两条指令之间应插入流水级数减一个空泡，与单周期无异。

为了避免以上情况的发生，现代 CPU 流水线在取指和译码时，都要对跳转指令进行分析，预测可能执行的分支和路径，防止预取错误的分支路径给流水线带来停顿。分支预测器：它会提前检测到跳转指令，并根据预判结果取指。如果预判结果是不跳转，就按顺序取下一条指令；如果预判结果是跳转，就从跳转的目的地址取下一条指令。假如

预测对了，那么流水线就不会被清空，仍然可以一条指令 1 个周期；如果预测错了，下一条指令仍然要 4 周期。从这里看出，分支预测器对于提高流水线效率有帮助。当然，以上情况并不适用于所有的内核，预判对错所需要的指令周期还是取决于 MCU 是如何设计的。

2.3.2 | 静态分支预测

其实，静态分支预测类似于不预测。常用静态分支预测一般有如下几种：

1. 总是不跳转：实现简单，无需跳转预测；对于未经优化的条件分支来说，准确率大概为 30% ～ 40%。
2. 总是跳转：实现简单，无需跳转预测；对于未经优化的条件分支来说，准确率大概为 60% ～ 70%。
3. BTFN（Backward taken, forward not taken）：如果目标地址比当前分支指令 PC 小，即往前跳，则跳转（如循环）；如果目标地址比当前分支指令 PC 大，即往后跳，则不跳。
4. Programmer based：程序员提供静态预测，通过编译器指令指示方向。`if (likely(x)) {....}` 指示较大概率是跳转，`if (unlikyly(error)) {....}` 指示较大概率是不跳转。

2.3.3 | 动态分支预测

动态预测则指在程序运行时进行预测。不同的软件、不同的程序分支行为，我们可以采取不同的算法去提高预测的准确率，如我们可以根据程序的历史执行路径信息来预测本次跳转的行为，常见的动态预测方式有 1-bit 动态预测、n-bit 动态预测、下一行预测、双模态预测、局部分支预测、全局分支预测、融合分支预测、循环预测等。随着大量新的应用软件的出现，为了应对新的程序逻辑行为，分支预测器也做得越来越复杂，占用的芯片面积也越来越大。在 CPU 内部，除了 Cache，就数分支预测器的电路版图最大。

分支预测技术是提高 CPU 性能的一项关键技术，其本质就是去除指令之间的相关性，让程序更高效运行。一个 CPU 性能高不高，不仅在于你的流水线有多深、主频有多高、Cache 有多大，还和分支预测技术息息相关。一个分支预测器好不好，我们可以从两个方面来衡量：分支判断速度和预测准确率。目前分支预测技术可以达到 95% 的预测准确率，然而技术进化之路永未停止，分支预测技术一直在随着计算机的发展不断更新迭代。

2.4 | 超标量处理器

2.4.1 | 为什么需要超标量

一个程序执行需要的总时长与总指令数、CPI 值和 CPU 频率有关。加快处理器执行程序的速度有以下三种策略:

1. **减少程序中指令的数量**。这取决于程序本身要完成的工作量、实现某个功能所选择的算法、编译器强大与否、指令集是否会对某些特殊的功能有扩展（如使用扩展的特殊指令集可以大大减少指令的数量）。
2. **减少每条指令在处理器中执行所需要的周期数**。对于非流水线的处理器，需要多个周期才能执行一条指令；对于普通的流水线处理器，每周期最多执行一条指令；对于超标量处理器，可以每周期执行多于一条的指令；再比如对于 VLIW（超长指令字）结构的处理器中每周期也可执行多于一条的指令。处理器采用何种架构实现称为微架构。
3. **减少处理器的时钟周期（即增大 CPU 的时钟频率）**。在进行处理器的微架构设计时，可以通过精巧的电路设计、更深的流水线来减少处理器的时钟周期，现代处理器的设计中，优秀的 EDA 工具也可帮助减少处理器的时钟周期。此外还有一个更关键的因素，即硅工艺。硅工艺的进步能够制造出更快更省电的处理器。

上述三种策略是彼此制约的。更小的 CPI 要求每周期执行更多条指令，这样会导致处理器的时钟周期很难降下来；而很小的时钟周期又很难容下复杂的逻辑设计，虽然可以通过更深的流水线来获得小的时钟周期，但却导致处理器在各种预测失败时有更大的惩罚，严重增大了处理器 CPI，并增大了功耗，造成“高频低能”的后果。

因此，超标量处理器为降低 CPI 而诞生，在超标量处理器中，每周期可从 I-Cache 中取出 n 条指令送到处理器的流水线中，处理器在每周期内也最少可以同时执行 n 条指令。

2.4.2 | 顺序执行的超标量处理器

顺序执行的超标量处理器是一种先进的处理器架构，它融合了并行处理与顺序执行的优势。在这种架构中，处理器能够在单个时钟周期内并行地从指令缓存中抓取多条指令，并对这些指令进行解码，识别出它们各自的操作类型、所需的操作数以及控制信号。这一过程极大地提高了指令的获取和准备效率。

解码后的指令随后被发送到多个专用的执行单元，如算术逻辑单元（ALU）、浮点单元（FPU）和乘法器等，以进行并行执行。这种并行执行机制使得超标量处理器能够同时处理多个计算任务，从而显著提高指令的吞吐率和程序的执行速度。

然而，尽管内部实现了高度的并行性，顺序执行的超标量处理器仍然坚持保持指令的原始执行顺序。这意味着，尽管指令在内部是并行处理的，但它们的结果将按照程序中指定的顺序被提交到寄存器堆中。这一特性确保了程序的逻辑正确性和执行结果的一致性，避免了由于并行执行可能导致的数据依赖和冲突问题。但是与之相对的，如果流水线中一个指令被阻塞，那么同级的其他指令也将一同被阻塞，知道对应所有指令全部被释放，才能进入到下一级。

总之，顺序执行的超标量处理器是一种集成了并行处理高效性和顺序执行稳定性的先进处理器架构，它通过优化指令的获取、解码、执行和提交过程，显著提高了程序的执行效率和性能表现。

2.4.3 | 乱序执行的超标量处理器

乱序执行的超标量处理器是现代计算机架构中的一个重要创新，它结合了超标量的并行处理能力和乱序执行的优化策略。首先，这种处理器能够在每个时钟周期内同时处理多条指令，通过增加执行单元的并行性来提升性能。这意味着，在处理器内部，多个指令可以同时被解码、分配到不同的执行单元，并几乎同时开始执行。

然而，与顺序执行的超标量处理器不同，乱序执行技术允许这些指令的执行顺序不再受限于原始的程序代码。处理器内部有一个复杂的调度器，它能够智能地分析指令间的依赖关系，并动态地调整指令的执行顺序。当某条指令的输入数据已经就绪，且其后继指令不依赖于其尚未完成的结果时，这条指令就可以被提前执行。这种灵活的执行方式大大减少了处理器的等待时间，提高了整体的执行效率。

乱序执行的超标量处理器的另一个关键优势在于其多发射能力。在顺序执行的超标量处理器中，尽管可以并行发射多条指令，但这些指令的执行和结果提交仍然受到原始顺序的约束。而在乱序执行的处理器中，由于执行顺序的灵活性，更多的指令可以同时进入执行阶段，只要它们之间的依赖关系得到妥善处理。这种高度的并行性使得处理器能够更高效地利用内部资源，进一步提升性能。

乱序执行的超标量处理器通过打破指令执行的原始顺序、优化指令间的依赖关系以及提升多发射能力，实现了更高的指令吞吐率和程序执行速度。

3 | CPU 核设计

3.1 | CPU 核心概述

项目使用了包括 Verilog，System Verilog 和 Python 脚本等多种语言，实现了一个支持部分 LA32R 指令集的单核五级流水单发射 CPU 核心。核心分为取指、译码、执行、访存、写回五级流水（标识为 s1 到 s5），通过 SRAM 接口与外部进行存储访问与读写。在指令集方面，核心支持基本的整数运算指令，包括算术运算类指令、移位运算类指令、转移指令和普通访存指令。处理器核成功通过了测试对应指令的龙芯官方提供测试点，这些测试点在 81 个总测试点中占据 46 个。另外，CPU 核心成功上板并通过测试，具体评估与总结一节。

3.2 | 流水线结构概述

CPU 流水线间使用 valid/ready 握手反压信号进行通信，并在其基础上实现了流水线的停顿和转发功能。CPU 设计接口对应 BRAM 类型同步存储器，故可以将指令存储器视为 CPU 的 s2 流水寄存器的一部分，将数据存储器视为 s4 流水寄存器的一部分。

在 s1 级，CPU 更新 PC 寄存器，并通过 SRAM 接口向指令存储器发送读取指令的地址。对于分支指令，PC 总是直接更新为 PC 自增 4 后的值，也即静态预测分支不跳转。如果在 s2 计算得出分支跳转，那么对应信号会将 s1 级 valid 置 0，从而使得 s1 级数据无效化。

在 s2 级，CPU 通过 SRAM 接口得到从 BRAM 读取的指令，并通过解码器解码出所有需要的控制信号。解码器参考 Lain Core 的部分代码，将所有指令、常量定义和控制信号均写入到若干个 json 文件中，然后通过 Python 脚本统一生成解码器文件和对应的接口/常量定义头文件。这样的定义方法很大程度上增加了代码的条理性，加快了开发速度，且能够较为便捷地进行修改，与龙芯官方提供的 openLA500 处理器中的解码方法相比具有较高的便捷性和先进性。另外，分支指令的分支结果也在本级计算得出，并传回 s1 来更新 PC 寄存器的值。

在 s3 级，CPU 通过 ALU 执行计算过程，其中 ALU 将乘除法器一同包括进去。另外对于访存指令，在 ALU 计算结束后会通过 SRAM 接口将访存地址和数据经处理后发出。

在 s4 级，CPU 通过 SRAM 接口接收访存数据，并进一步处理。

在 s5 级，处理器更新相关数据信号并写回寄存器堆。

对于流水线的数据前递，处理器中采取了控制信号与阻塞信号结合的方式进行实现。每条指令均会在解码时解码出最早产生写入数据的流水级，用于停顿和转发的信号比较用。若指令在某一级结束时产生有效的写入寄存器数据，则在对应流水级信号为高

位，此时如果存在某一级需要这级流水线的数据，在本拍结束时这个信号不仅会写入到下一级流水中，也会写入到对应转发目标的流水线寄存器中，从而实现流水线的有效转发。

4 | SoC 设计及系统适配

4.1 | SoC 设计

项目围绕 CPU 核心，针对龙芯体系结构教学实验箱（Artix-7）设计了支持部分板上硬件的 SoC。部分龙芯开发板上硬件列举如表 1 。

功能模块	设计方案概述
FPGA	选用 Artix-7 XC7A200T-FBG676
DDR3	使用 FPGA 实现 DDR3 控制器，板载 K4B1G1646G-BCK0 DDR3 颗粒
SRAM	使用 FPGA 实现 SRAM 控制器，板载 IDT71V124SATY SRAM 芯片
NAND	使用 FPGA 实现 NAND 控制器，板载 K9F1G08U0C-PCB0 闪存颗粒
SPI Flash 1	使用 FPGA 实现 SPI 控制器（支持启动），板载 Flash 芯片插座，Flash 可插拔
SPI Flash 2	板载不可插拔 Flash 芯片，FPGA 设计固化专用
VGA	使用 FPGA 实现数字显示模块，板载 MM74HC573SJ 实现 332 的数模转换，模拟 VGA 的 R、G、B 信号
LCD	使用 FPGA 实现 LCD 显式控制器，板载 TFT-LCD 屏
USB	使用 FPGA 实现 USB 控制器，板载 USB PHY（USB3500），对外提供一个 USB 接口
LAN	使用 FPGA 实现 MAC 控制器，板载以太网 PHY（DM9161AEP），对外提供一个 RJ45 网络接口
PS2	使用 FPGA 实现 PS2 控制器，板载 PS2 接口
UART	使用 FPGA 实现 UART 控制器，板载 UART 接口
GPIO	16 个 LED 单色灯；2 个 LED 双色灯；8×8 LED 点阵（可实现字符显示功能）；8 个共阴极八段数码管（用于数字显示）；其余外接通用 I/O 接口。

表 1　龙芯 Artix 开发板上的主要部件

项目设计的 SoC 参考龙芯官方的板载外设参数寄存器代码，将板载外设映射到部分存储器空间上，以实现中央处理器对外设的调用。在 SoC 中，处理器的指令 SRAM 接口直接连接到一个 BRAM 存储器上，而处理器的数据 SRAM 接口通过一个 1x2 的 SRAM Bridge 分别连接到一个数据 BRAM 存储器和外设参数存储器上，以实现访问不同空间能够分别访问数据存储器和外设。

关于时钟方面，板载时钟提供 100 MHz 的时钟源。为了更好地适应 SoC 调试的需要，在 SoC 中添加了 Xilinx IP 核 PLL 组件，将外置时钟转化为 10 MHz 时钟以便于调试。

关于外设方面，SoC 主要实现了对 GPIO 接口的支持，也即支持对主板上 16 个单色 LED 灯，2 个双色 LED 灯，8×8 LED 点阵和 8 个阴极八段数码管和相关控制按钮，拨码开关的支持。

关于地址映射方面，外设寄存器中具体地址映射如表 2 。

地址映射标识	映射地址	地址映射标识	映射地址
CR0_ADDR	32'hbfaf_8000	CR1_ADDR	32'hbfaf_8010
CR2_ADDR	32'hbfaf_8020	CR3_ADDR	32'hbfaf_8030
CR4_ADDR	32'hbfaf_8040	CR5_ADDR	32'hbfaf_8050
CR6_ADDR	32'hbfaf_8060	CR7_ADDR	32'hbfaf_8070
LED_ADDR	32'hbfaf_f020	LED_RG0_ADD	32'hbfaf_f030
LED_RG1_ADD	32'hbfaf_f040	NUM_ADDR	32'hbfaf_f050
SWITCH_ADD	32'hbfaf_f060	BTN_KEY_ADD	32'hbfaf_f070
BTN_STEP_ADD	32'hbfaf_f080	SW_INTER_ADD	32'hbfaf_f090
TIMER_ADDR	32'hbfaf_e000	IO_SIMU_ADDR	32'hbfaf_ff00
VIRTUAL_UART_ADD	32'hbfaf_ff10	SIMU_FLAG_ADDR	32'hbfaf_ff20
OPEN_TRACE_ADD	32'hbfaf_ff30	NUM_MONITOR_ADD	32'hbfaf_ff40

表 2　SoC 中外设参数寄存器的主要地址映射关系

4.2 | 系统软件移植

项目尝试进行了部分软件移植，但是限于处理器开发进度，移植到 LA32R 指令集的软件并没有在处理器上进行测试，而仅在龙芯官方提供的虚拟机 QEMU 上进行了相关测试。

项目中 U-Boot 引导程序以 la32r-uboot 为基本，添加 `la32rsoc_defconfig` 配置，并通过添加 `configs/la32rsoc_defconfig`、`include/configs/la32rsoc_diy.h` 以及 `arch/la32r/dts/la32rsoc_diy.dts` 文件以及修改 `arch/la32r/Kconfig`、`arch/la32r/cpu/start.S`、`arch/la32r/dts/Makefile`、`board/nscscc/Kconfig` 文件添加对本处理器 Soc 的单独的针对性配置。

使用 `loongarch32r-linux-gnusf-` 工具链进行编译。具体编译命令如下（在 u-boot 根目录下）

```
export ARCH=la32r
export CROSS_COMPILE=loongarch32r-linux-gnusf-
make la32rdiy_defconfig
make
loongarch32r-linux-gnusf-objdump -S u-boot > u-boot.S
loongarch32r-linux-gnusf-objcopy ./u-boot -O binary u-boot.bin
```

调试后的程序可以在 `qemu-system-loogarch32r` 上正常运行。

5 | 评估及总结

5.1 | Soc 仿真及上板情况

Soc 在 Xilinx Vivado 2023.2 版本中进行仿真，通过了龙芯官方对于 LA32R 指令集设计的 81 个测试点中的 46 个，也即所有的项目实现的功能对应的测试点都已通过。具体项目文件见结题材料，仿真日志如下。

```
Test begin!
----[  12455 ns] Number 8'd01 Functional Test Point PASS!!!
        [  22000 ns] Test is running, debug_wb_pc = 0x1c04c0e4
        [  32000 ns] Test is running, debug_wb_pc = 0x1c04d084
----[  36915 ns] Number 8'd02 Functional Test Point PASS!!!
        [  42000 ns] Test is running, debug_wb_pc = 0x1c061a5c

...... Logs Omitted ......

----[1200555 ns] Number 8'd45 Functional Test Point PASS!!!
        [1202000 ns] Test is running, debug_wb_pc = 0x1c06ff8c
        [1212000 ns] Test is running, debug_wb_pc = 0x1c070dac
        [1222000 ns] Test is running, debug_wb_pc = 0x1c071bd4
----[1222565 ns] Number 8'd46 Functional Test Point PASS!!!
================================================================
Test end!
----PASS!!!
```

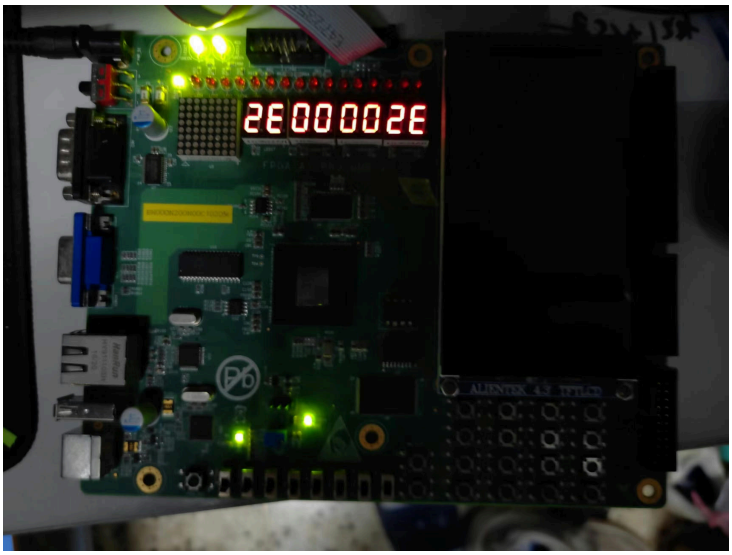

图 6　将 SoC 综合后刷入龙芯开发板

经过调试后，成功将 SoC 进行综合并实现，然后生成二进制流文件并刷入龙芯开发板载 FPGA。为调试方便，上板的频率设置为 10 MHz，尚未对时序做出优化。上板并观察到所有红色 LED 灯全灭，两个双色 LED 灯为绿色，意味着通过 46 个测试点，对应实物图如图 6。综合后的时序分析数据见图 7 。

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	11.208 ns	Worst Hold Slack (WHS):	0.121 ns	Worst Pulse Width Slack (WPWS):	2.633 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	7343	Total Number of Endpoints:	7343	Total Number of Endpoints:	1833

All user specified timing constraints are met.

图 7　实现后的时序分析

5.2 | 项目总结

项目基于 LA32R 指令集的一个功能性子集实现了一个简单的五级流水线处理器，帮助项目成员们一定程度上了解了 LA32R 指令集的构成和特性。另外在完成处理器和对应 SoC，以及实现对应软件移植的过程中，项目成员们也积累了一定的相关工作经验，并最终通过了对应处理器的功能测试点并通过上板验证。

但是项目也存在许多不足之处。项目组部分成员参与“龙芯杯”系统能力竞赛，但是对应分工并不全面，不足以支持开发出一颗较为功能全面的处理器核。组中两个成员分别负责处理器分支预测部分的设计和后端流水线的设计，这与五级流水线处理器的功能实现关联性并不大，也导致了在开发过程中出现了一系列困难。但是在开发本颗核心的过程中同学们仍旧经历了一个相对完整的核心开发过程，并得到了一定的成果，这对日后参加“龙芯杯”竞赛想必应当具有一定的帮助。

感谢助教王哲、苏阳和老师万寒、杨建磊在课程开发过程中提供的帮助和指导。

参考文献

[1] 龙芯架构 32 位精简版参考手册 V1.03[M]. 龙芯中科技术股份有限公司.

[2] 刘明道, 高焕昂, 王博文, 等. NOP: A Strong Baseline for LoongArch Out-of-order Processor[R].

[3] NSCSCC-openLA500[EB/OL]. 龙芯中科技术股份有限公司. https://gitee.com/wang-zhaoxiong/nscscc-openla500?skip_mobile=true.

[4] “流云”处理器[EB/OL]. https://gitee.com/UCAS-Muradil/LiuYun.

[5] “卅”处理器[EB/OL]. https://gitee.com/MJ_Wang/spinal-loong-arch-core/tree/master.

[6] “争流”处理器[EB/OL]. https://gitee.com/liangliang678/ZhengLiu.

[7] nontrivial-mips[EB/OL]. https://github.com/trivialmips/nontrivial-mips/.

[8] LLCL-MIPS[EB/OL]. https://github.com/huang-jl/LLCL-MIPS/.

[9] zencove-zoom[EB/OL]. https://github.com/zencove-thu/zencove-zoom.

[10] VexRiscv[EB/OL]. https://github.com/SpinalHDL/VexRiscv.

[11] MITTAL S. A survey of techniques for dynamic branch prediction[J/OL]. Concurrency and Computation: Practice and Experience, 2019, 31(1): e4666[2024-05-20]. https://onlinelibrary.wiley.com/doi/abs/10.1002/cpe.4666.

[12] PERAIS A, SHEIKH R. Branch Target Buffer Organizations[C/OL]//Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture. New York, NY, USA: Association for Computing Machinery, 2023: 240-253[2024-05-20]. https://dl.acm.org/doi/10.1145/3613424.3623774.

[13] 颜伟业, 耿琛博, 杜鑫, 等. 马家沟河项目决赛设计报告[R].

[14] Lain Core[EB/OL]. 北京航空航天大学. https://github.com/LainChip/complex_loong_cpu.

[15] FDU1.1[EB/OL]. 复旦大学. https://github.com/NSCSCC-2020-Fudan/FDU1.1-NSCSCC.